01

# Part 1 · Cloud Foundations

What the cloud is, why companies move to it, how it is paid for, and who is responsible for what.

- Server vs Client
- Cloud Computing
- On-Prem vs Cloud · CapEx vs OpEx
- Pay-as-you-go
- Public · Private · Hybrid
- IaaS · PaaS · SaaS
- Cloud Service Providers
- Shared Responsibility Model

# 1.1 Server vs Client

★★☆

**IN ONE LINE**

A client requests a service; a server is an always-on computer that provides it.

Almost everything on the internet works as a **conversation**: a **client** sends a request, a **server** sends back a response. A "server" is just a computer (or a program on it) that is switched on all the time and offers a service to many clients.

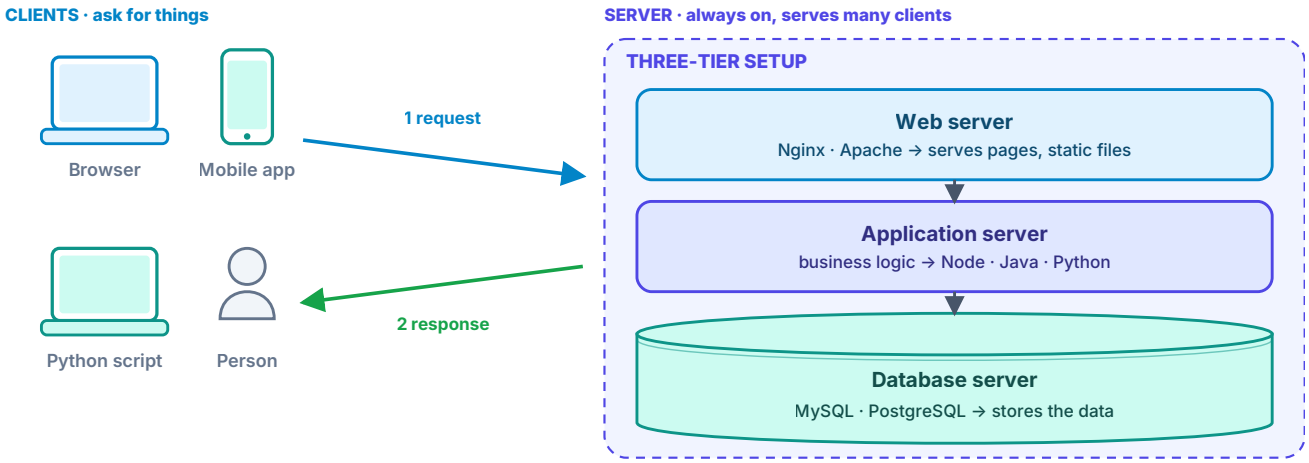



Figure 1.1 — Clients send requests; servers respond

| | Client | Server |
|---|---|---|
| **Role** | Asks for a service | Provides the service |
| **Examples** | Browser, mobile app, `curl`, a script | Web server, database server, mail server, file server |
| **Lifetime** | Starts when the user needs it | Runs 24 × 7, waits for requests |
| **Hardware** | Laptop, phone | Powerful machine in a data centre — or a cloud VM |

**REAL-WORLD ANALOGY**

A restaurant: **you are the client** (you order), the **kitchen is the server** (it prepares and serves, for many tables at once). The waiter is the network.

**INTERVIEW ANGLE**

"Server" means two things: the **physical machine** and the **software** running on it (e.g. an Nginx web server). In the cloud you rent the machine as a virtual server, and you install or use the software on top.

# 1.2 Cloud Computing

★★★

**IN ONE LINE**

Renting computing resources over the internet, on demand, and paying only for what you use.

**Cloud computing** is the on-demand delivery of IT resources — servers, storage, databases, networking, software, analytics — over the internet with pay-as-you-go pricing. Instead of buying and running your own hardware, you **rent** it from a provider who runs huge data centres.

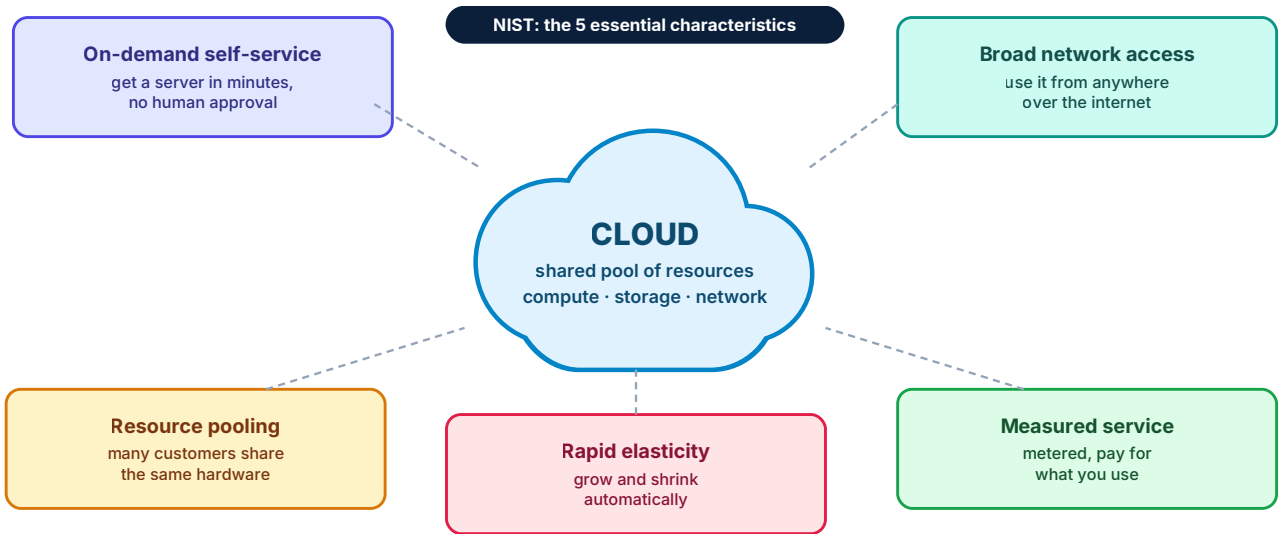
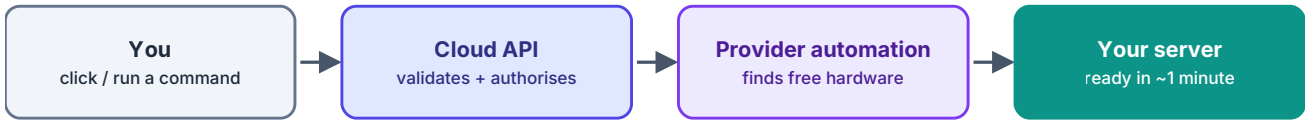



Figure 1.2a — The five essential characteristics of cloud (NIST definition)




No purchase order, no shipping, no rack-and-stack: the whole flow is automated.

Figure 1.2b — What "on-demand" means in practice

### Why companies move to cloud

- **Speed** — experiment in minutes, not months.
- **Cost model** — no big upfront spend.
- **Global reach** — deploy near users worldwide.
- **Reliability** — built-in redundancy.
- **Managed services** — the provider runs databases, ML, analytics for you.

### Trade-offs to be honest about

- Ongoing bills can grow if unmanaged.
- Less control over underlying hardware.
- Security is **shared** (see 1.9).
- Dependence on the provider (vendor lock-in, 8.2).
- Needs new skills (IAM, networking, cost control).

**REAL-WORLD ANALOGY**

Electricity: you do not build a power plant, you plug in and pay for the units you use. Cloud is the same for computing.

# 1.3 On-Premises vs Cloud

★★★

**IN ONE LINE**

Owning and running your own hardware versus renting it from a provider.

**On-premises (on-prem)** means the company owns and operates its servers in its own building or leased space. **Cloud** means renting equivalent capacity from a provider. Most real companies today use both (see hybrid, 1.6).

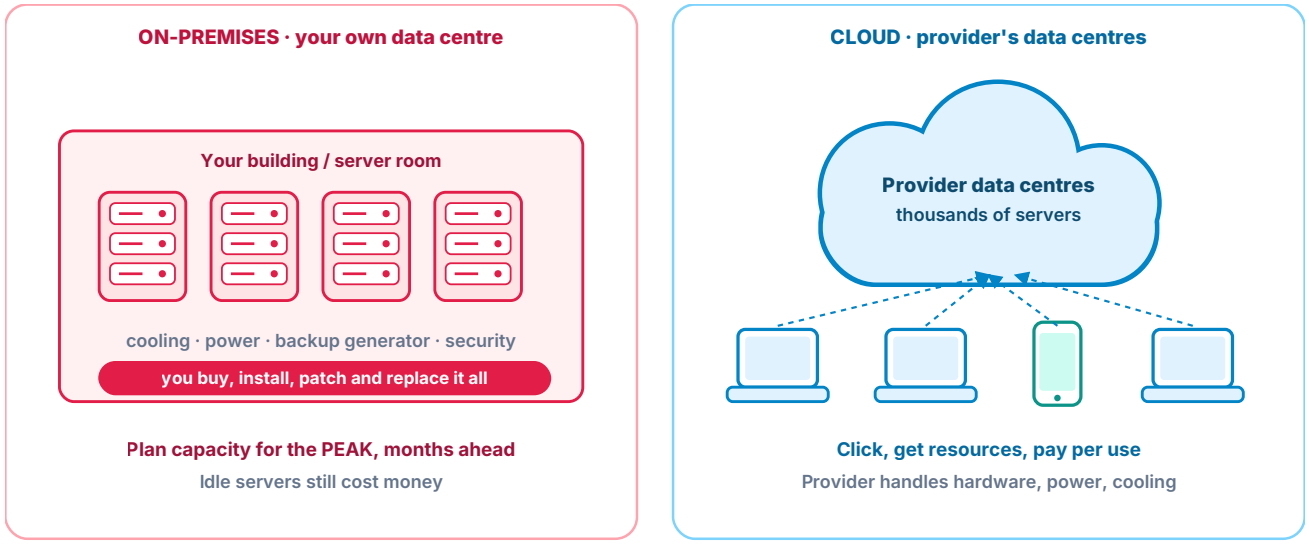



Figure 1.3 — Who owns and runs the hardware?

| | On-premises | Cloud |
|---|---|---|
| **Upfront cost** | High (buy servers, build the room) | Near zero |
| **Setup time** | Weeks to months (procurement, install) | Minutes |
| **Scaling** | Buy more hardware; slow, lumpy | Instant, automatic, granular |
| **Maintenance** | Your team: patches, repairs, upgrades | Provider handles hardware and facility |
| **Control** | **Full** control of hardware and data location | Shared; you control configuration, not the hardware |
| **Security / compliance** | You are fully responsible | Shared responsibility (1.9) |
| **Cost shape** | Fixed, whether used or not | Variable, follows usage |
| **Good fit** | Strict data-residency rules, stable predictable load, legacy systems | Variable load, fast growth, startups, global apps, experiments |

**INTERVIEW ANGLE**

Do not answer “cloud is always better”. A balanced line: **cloud wins on agility and variable workloads; on-prem can win on cost for steady, predictable, long-running workloads or when regulations demand physical control.**

# 1.4 CapEx vs OpEx

★★★

**IN ONE LINE**

Capital expenditure buys assets up front; operating expenditure pays for usage as it happens.

Two ways of paying for IT. **CapEx** (capital expenditure) is a large up-front purchase of an asset that is used for years. **OpEx** (operating expenditure) is a recurring cost for running the business. Cloud turns IT from CapEx into OpEx.

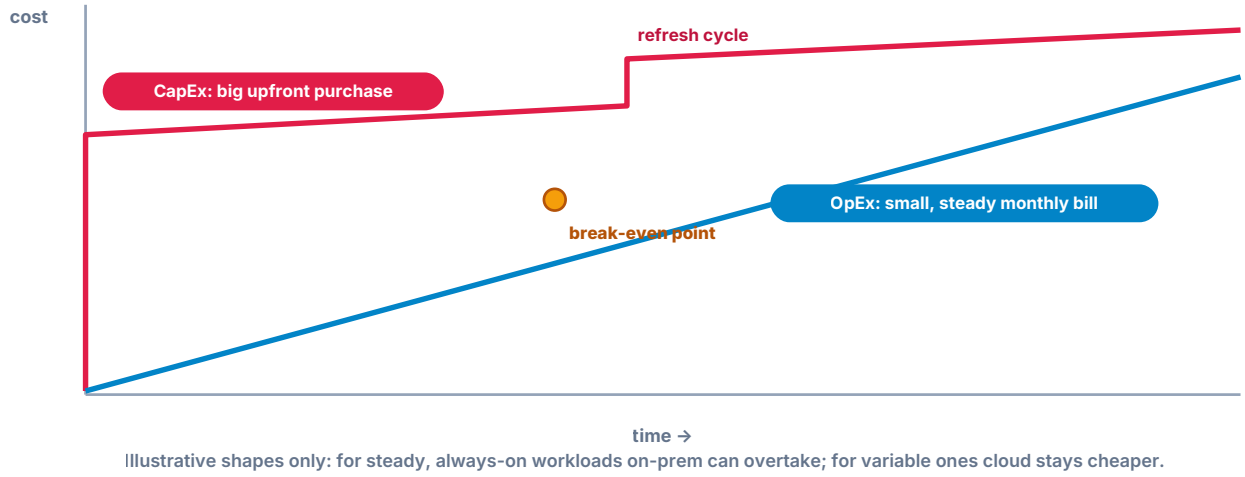



Figure 1.4 — Cost shape over time: up-front purchase vs pay-as-you-go

| | CapEx | OpEx |
|---|---|---|
| **Meaning** | Buy assets: servers, network gear, building | Pay for services as you consume them |
| **Cash flow** | Large payment up front | Smaller regular payments |
| **Accounting** | Capitalised and **depreciated** over years | Expensed in the period incurred |
| **Risk** | You carry it: over-buy and waste, under-buy and run out | Provider carries capacity risk |
| **Flexibility** | Locked in for the asset's life | Stop paying when you stop using |
| **Typical example** | Buying 50 servers for a data centre | Monthly cloud bill, SaaS subscription |

**REAL-WORLD ANALOGY**

Buying a car (CapEx: big payment, you maintain it, it loses value) vs using a ride-hailing app (OpEx: pay per trip, no maintenance).

**COMMON TRAP**

OpEx is not automatically cheaper. A busy, always-on workload can cost more in the cloud than owned hardware. The real benefit is **flexibility and lower risk**, which is why finance teams like OpEx for uncertain projects.

# 1.5 Pay-as-you-go Pricing

★★★

**IN ONE LINE**

Billing based on actual consumption, metered by the second, GB or request.

In **pay-as-you-go** (utility) pricing you are billed for what you actually consume — seconds of compute, GB-months of storage, GB of data transferred, number of requests — instead of a flat fee for capacity you may not use.




Figure 1.5 — Fixed capacity vs metered usage, and the common purchase options

| What is metered | Typical unit | Example |
|---|---|---|
| **Compute** | per second / hour a VM runs | A VM running 10 hours costs for 10 hours |
| **Storage** | per GB per month | 500 GB in object storage for a month |
| **Data transfer** | per GB **leaving** the cloud (egress) | Inbound is usually free; outbound is charged |
| **Requests / executions** | per million requests | Serverless function invocations, API calls |
| **Managed service** | per hour or per unit of throughput | A database instance, query bytes scanned |

**COMMON TRAP**

"Pay for what you use" also means **pay for what you forget to turn off**. A forgotten VM, an unattached disk or a public-facing data egress can quietly build a large bill. Set budgets and alerts from day one (see cost optimisation, 8.1).

**INTERVIEW ANGLE**

Know the three levers of the bill: **compute time**, **storage volume**, and **data egress**. Reserved or savings commitments suit steady workloads; spot capacity suits fault-tolerant batch jobs such as Spark ETL.

# 1.6 Public vs Private vs Hybrid Cloud

★★★

**IN ONE LINE**

Deployment models: who owns the infrastructure, and who shares it.

These describe **who the infrastructure belongs to and who shares it** — the deployment model.

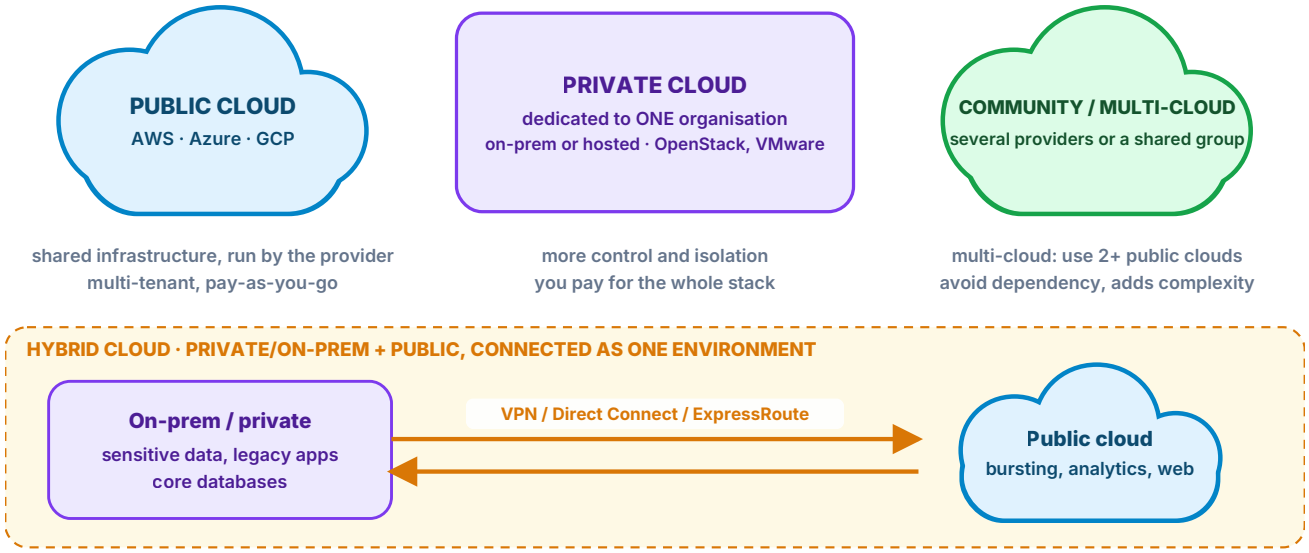



Figure 1.6 — Public, private, hybrid (and multi-cloud)

| | Public cloud | Private cloud | Hybrid cloud |
|---|---|---|---|
| **Owner / operator** | Cloud provider | One organisation (or hosted exclusively for it) | Mix of both, connected |
| **Who shares it** | Many customers (multi-tenant) | Only you | Depends on the part |
| **Cost** | Pay-as-you-go, no capex | High: hardware + staff | Balanced |
| **Scalability** | Practically unlimited | Limited to your hardware | Burst into the public side |
| **Control / compliance** | Lower (shared model) | **Highest** | Keep regulated data private |
| **Example use** | Startups, web apps, analytics | Banks, defence, healthcare records | Core DB on-prem, analytics and web in cloud |

**INTERVIEW ANGLE**

**Multi-cloud** (using AWS *and* Azure) is different from **hybrid** (on-prem *and* cloud). Interviewers like checking that you know the difference. Real-life reasons for hybrid: data-residency laws, existing investments, low-latency to factory floors, gradual migration.

# 1.7 IaaS vs PaaS vs SaaS

★★★

**IN ONE LINE**

Service models that differ in how much of the technology stack the provider manages.

The three **service models** describe **how much of the stack the provider manages for you**. Moving from IaaS to SaaS you give up control and get less operational work.

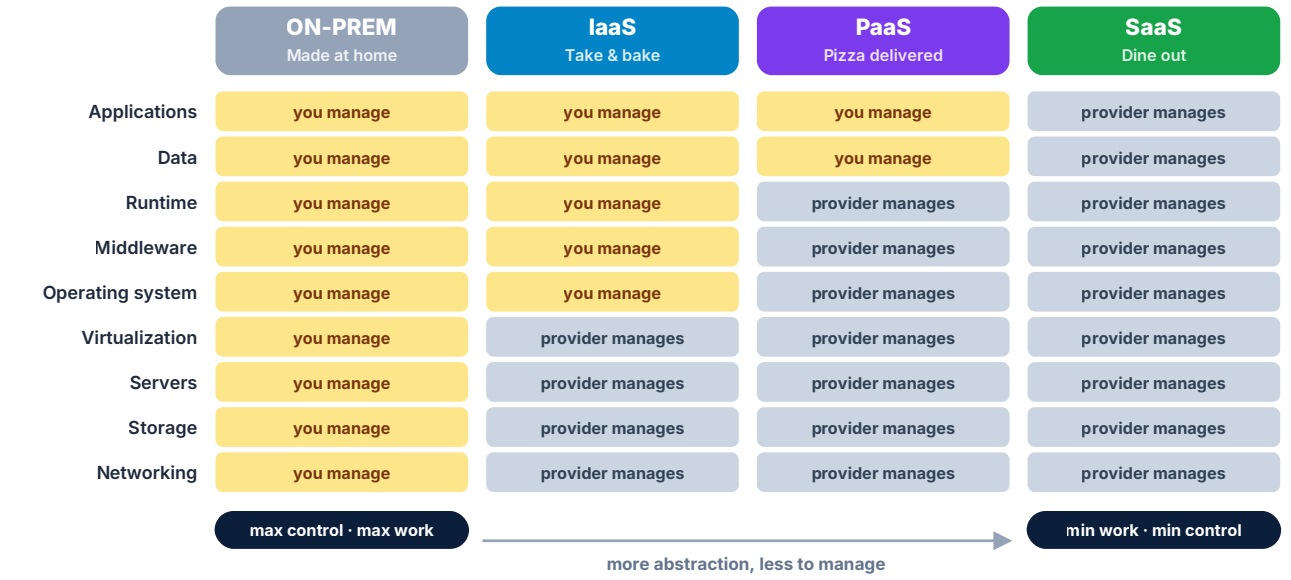



Figure 1.7a — "Pizza as a Service": who manages which layer

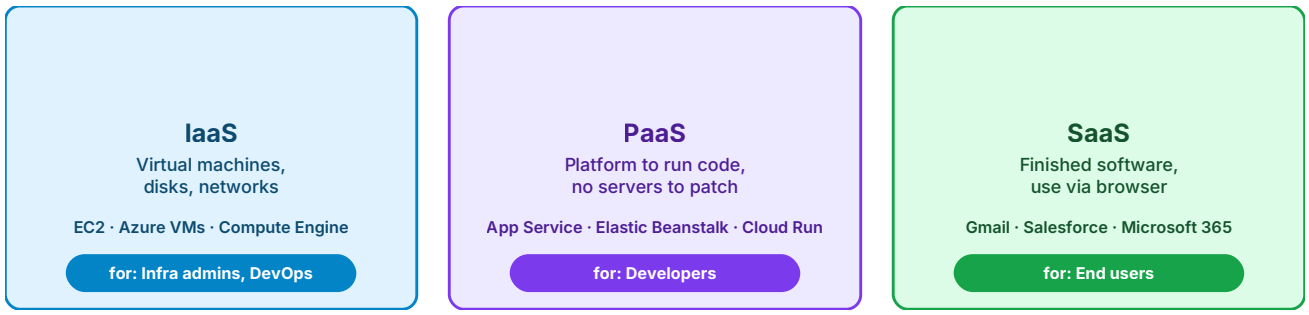



Figure 1.7b — The three models with examples

| | IaaS | PaaS | SaaS |
|---|---|---|---|
| **You get** | Raw building blocks: VMs, storage, networks | A ready platform to deploy code or data | A complete application |
| **You manage** | OS, runtime, apps, data, security config | Your application and data | Only your usage / settings |
| **Provider manages** | Hardware, virtualisation, facilities | + OS, runtime, scaling, patching | Everything |
| **Flexibility** | Highest | Medium | Lowest |
| **Examples** | EC2, Azure VM, Compute Engine, S3 | Heroku, Azure App Service, Cloud Run, managed databases | Gmail, Slack, Salesforce, Google Workspace |
| **Persona** | Sysadmin / DevOps | Developer / data engineer | Business user |

**INTERVIEW ANGLE**

Other "aaS" terms to recognise: **FaaS** (functions as a service — serverless, 2.6), **DBaaS** (managed databases), **CaaS** (containers as a service). Be ready to **classify examples**: S3 = IaaS-style storage service, Snowflake = SaaS/PaaS-style data platform, Databricks = PaaS/SaaS depending on view — say your reasoning, not just the label.

# 1.8 Cloud Service Provider (CSP)

★★☆

**IN ONE LINE**

A company that runs global data centres and rents computing, storage and services to customers.

A **Cloud Service Provider (CSP)** is a company that owns data centres around the world and rents out computing services over the internet. The "big three" are **AWS, Microsoft Azure and Google Cloud**.

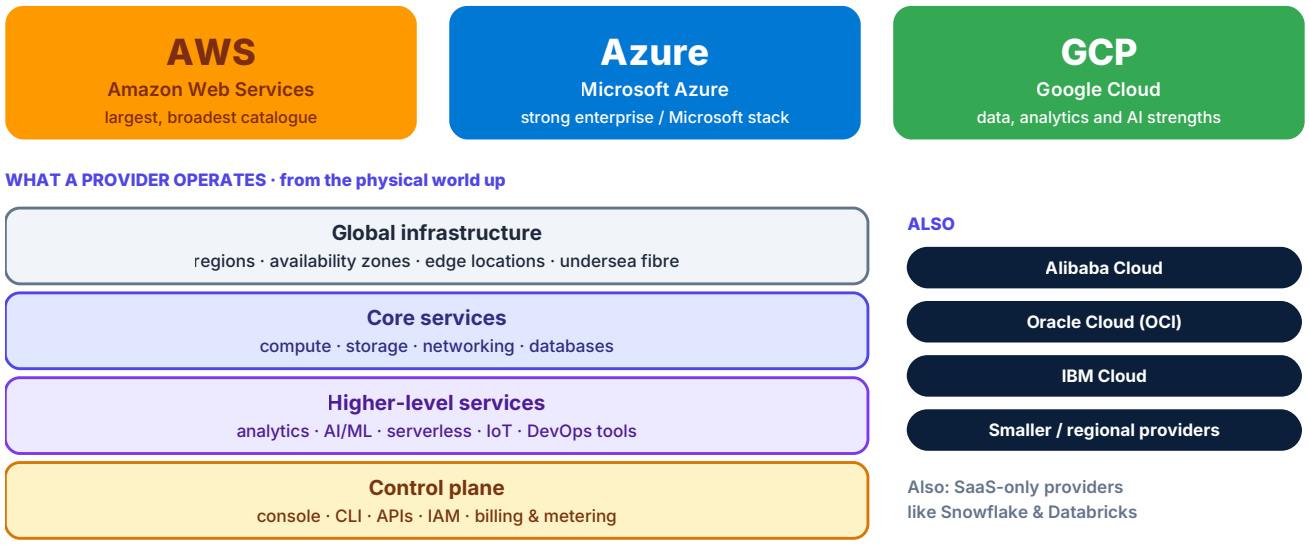



Figure 1.8 — The major providers and what they run

| Category | AWS | Azure | Google Cloud |
|---|---|---|---|
| **Virtual machines** | EC2 | Virtual Machines | Compute Engine |
| **Object storage** | S3 | Blob Storage | Cloud Storage |
| **Serverless functions** | Lambda | Functions | Cloud Functions / Cloud Run |
| **Managed Kubernetes** | EKS | AKS | GKE |
| **Identity & access** | IAM | Entra ID + Azure RBAC | Cloud IAM |
| **Data warehouse** | Redshift | Synapse / Fabric | BigQuery |

**INTERVIEW ANGLE**

Services are **equivalent across providers with different names**. Learn one provider deeply and map the concepts to the others — the full cheat sheet is in 8.6. At the fresher level, interviewers care that you understand the *concept* (object storage, VM, serverless), not memorised brand names.

### How you talk to a CSP

- **Web console** (point and click)
- **CLI** ( `aws` , `az` , `gcloud` )
- **SDKs** (Python `boto3` , etc.)
- **Infrastructure as Code** (Terraform, 7.5)

### Choosing a provider

- Existing skills and contracts
- Services you need (e.g. AI, analytics)
- Regions and compliance coverage
- Pricing and support
- Ecosystem and community

## 1.9 Shared Responsibility Model

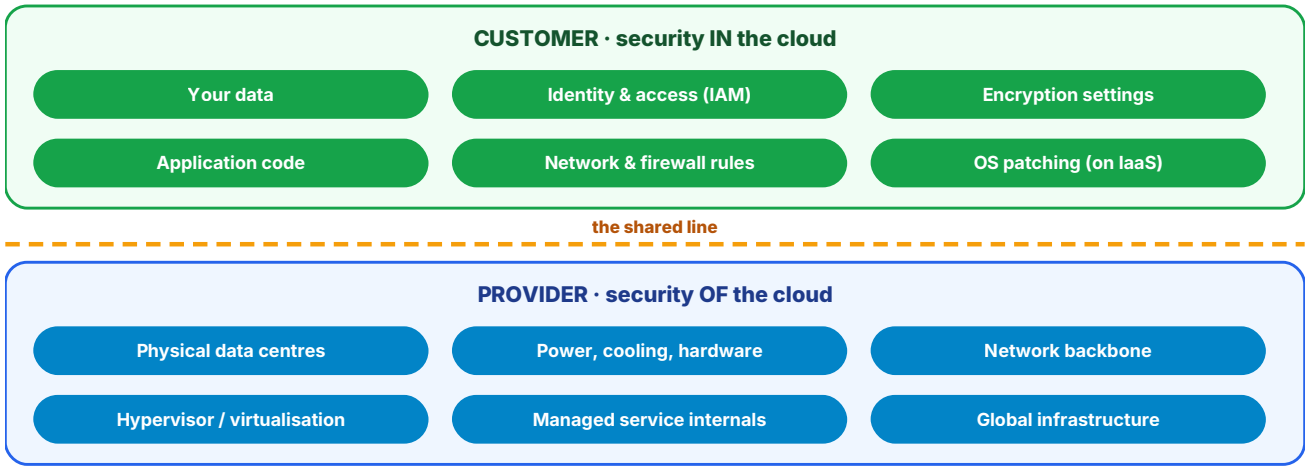
★★★

**IN ONE LINE**

The provider secures the infrastructure; you secure your data, access and configuration.

Security in the cloud is a **partnership**. The provider secures the cloud itself; **you** secure what you put in it and how you configure it. Most real-world cloud breaches are caused by **customer misconfiguration** (e.g. a public bucket, an over-permissive role), not by the provider being hacked.

**WHO IS RESPONSIBLE FOR SECURITY?**




The line moves up as you go from IaaS to PaaS to SaaS (the provider does more).

Figure 1.9a — Security OF the cloud vs security IN the cloud

| | Data & access | Application | OS & runtime | Virtualisation | Physical / network |
|---|---|---|---|---|---|
| On-prem | customer | customer | customer | customer | customer |
| IaaS | customer | customer | customer | customer | provider |
| PaaS | customer | customer | provider | provider | provider |
| SaaS | customer | provider | provider | provider | provider |

Figure 1.9b — The split changes with the service model

| Example | Who is responsible? |
|---|---|
| **A fire in the data centre; failed disk hardware** | **Provider** |
| **Patching the hypervisor** | **Provider** |
| **Patching the OS on your EC2 / Azure VM** | **You** (IaaS) |
| **Patching the OS under a managed database or serverless function** | **Provider** (PaaS / serverless) |
| **Who is allowed to read your S3 bucket** | **You** (always) |
| **Enabling MFA and strong passwords for your users** | **You** (always) |
| **Encrypting your data and managing the keys** | **You** choose and configure; provider supplies the tools |

**REMEMBER**

A simple rule: **the provider is responsible for the cloud; the customer is responsible for everything they put in it** — always your data, identities, access settings and configuration.